Lec 07 Temporal-Difference Learning

TD Learning of state values
- Algorithm
- TD target and TD value
- Other Properties:

TD learning of action values - Sarsa (state-action-reward-state-action)
- Sarsa - algorithm
- Pseudocode

TD learning of action values - Expected Sarsa

TD learning of action values - n-step Sarsa

TD learning of optimal action values: Q-learning
- **Q-learning algorithm**
- Off-policy & On-policy
  - Advantages of off-policy learning:
  - How to judge
- Pseudocode
  - Off - policy
  - On - Policy

A unified point of view

TD Learning of state values

Algorithm

Given a policy π, the aim is to estimate the state values $v_\pi(s)$ under π.

The data/experience required by the algorithm:

$(s_0, r_1, s_1, \ldots, s_t, r_{t+1}, s_{t+1}, \ldots)$ or $\{(s_t, r_{t+1}, s_{t+1})\}_t$ generated following the given policy π.

The *TD learning* algorithm is

$$\underbrace{v_{t+1}(s_t)}_{\text{new estimate}} = \underbrace{v_t(s_t)}_{\text{current estimate}} - \alpha_t(s_t)\underbrace{[v_t(s_t) - \overbrace{[r_{t+1} + \gamma v_t(s_{t+1})]}^{\text{TD target } \bar{v}_t}]}_{\text{TD error } \delta_t} \quad (1)$$

$$v_{t+1}(s) = v_t(s), \quad \forall s \neq s_t, \quad (2)$$

where $t = 0, 1, 2, \ldots$

- Here, $v_t(s_t)$ is the estimated state value of $v_\pi(s_t)$;$\alpha_t(s_t)$ is the learning rate of s_t at time t.
- At time t, only the value of the visited state s_t is updated whereas the values of the unvisited states $s \neq s_t$ remain unchanged.
- The update in (2) will be omitted when the context is clear.

TD target and TD value

It is clear that the new estimate $v_{t+1}(s_t)$ is a combination of the current estimate $v_t(s_t)$ and the **TD error**.

Interpretation of **TD target** $\bar{v}_t$:

The algorithm drives $v(s_t)$ towards $\bar{v}_t$.

$$\begin{aligned} v_{t+1}(s_t) &= v_t(s_t) - \alpha_t(s_t)[v_t(s_t) - \bar{v}_t] \\ \implies vt+1(s_t) - \bar{v}_t &= v_t(s_t) - \bar{v}_t - \alpha_t(s_t)[v_t(s_t) - \bar{v}_t] \\ \implies vt+1(s_t) - \bar{v}_t &= [1 - \alpha_t(s_t)][v_t(s_t) - \bar{v}_t] \\ \implies |vt+1(s_t) - \bar{v}_t| &= |1 - \alpha_t(s_t)||v_t(s_t) - \bar{v}_t| \end{aligned}$$

Since $\alpha_t(s_t)$ is a small postive number, we have $0 < 1 - \alpha_t(s_t) < 1$

Therefore, $|v_{t+1}(s_t) - \bar{v}_t| \leq |v_t(s_t) - \bar{v}_t|$, which means $v(s_t)$ is driven towards $\bar{v}_t$!

Interpretation of **TD error** δ_t:

It is a difference between two consequent time steps and it also reflects the deficiency between v_t and v_π, donate that

$$\begin{gathered} \delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})] \\ \mathbb{E}[\delta_{\pi,t}|S_t = s_t] = v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s_t] = 0. \end{gathered}$$

If $v_t = v_\pi$, then δ_t should be zero (in the expectation sense). Hence, if δ_t is not zero, then v_t is not equal to v_π.
The TD error can be interpreted as **innovation**, which means new information obtained from the experience (s_t, r_{t+1}, s_{t+1}).

Other Properties:

The TD algorithm only estimate the state value of a given policy, it does not estimate the action values and does not search for optimal policies.

TD learning of action values - Sarsa (state-action-reward-state-action)

Sarsa - algorithm

The aim is to estimate the action values of a given policy π.

Suppose we have some experience $\{(s_t, a_t, r_{t+1}, s_{t_1}, a_{t+1})\}_t$

We can use the following ***Sarsa*** algorithm to estimate the action values:

$$\begin{aligned} q_{t+1}(s_t, a_t) &= q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\right], \\ q_{t+1}(s, a) &= q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t), \end{aligned}$$

where $t = 0, 1, 2, \ldots$

- $q_t(s_t, a_t)$ is an estimate of $q_\pi(s_t, a_t)$;
- $\alpha_t(s_t, a_t)$ is the learning rate depending on s_t, a_t.

> The relation ship between Sarsa and previous TD learning algorithm:
> We can obtain Sarsa by replacing the state value estimate $v(s)$ in the TD algorithm with the action value estimate $q(s, a)$. **As a result, Sarsa is an action-value version of the TD algorithm.**

The expression of Sarsa suggests that it is a stochastic approximation algorithm solving the following equation:

$$q_\pi(s, a) = \mathbb{E}[R + \gamma q_\pi(S', A')|s, a], \quad \forall s, a.$$

This is **another expression of the Bellman equation expressed in terms of action values**.

Pseudocode

The ultimate goal of RL is to find optimal policies. To do that, we can combine Sarsa with a policy improvement step. The combined algorithm is also called Sarsa.

Pseudocode: Policy searching by Sarsa

For each episode, do

If the current s_t is not the target state, do

Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}, and then take action a_{t+1} following $\pi_t(s_{t+1})$.

Update q-value:

$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - [r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})]\Big]$

Update policy:

$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}|}(|\mathcal{A}| - 1)$ if $a = \arg\max_a q_{t+1}(s_t, a)$

$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}|}$ otherwise

TD learning of action values - Expected Sarsa

A variant of Sarsa is the Expected Sarsa algorithm:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - (r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)])\right],$$
$$q_{t+1}(s, a) = q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),$$

where

$$\mathbb{E}[q_t(s_{t+1}, A)] = \sum_a \pi_t(a|s_{t+1}) q_t(s_{t+1}, a) \doteq v_t(s_{t+1})$$

is the expected value of $q_t(s_{t+1}, a)$ under policy π_t.

Compared to Sarsa:

- **The TD target is changed** from $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$as in Sarsa to $r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1}, A)]$ as in Expected Sarsa.
- Need more computation. But it is beneficial in the sense that it reduces the estimation variances because it reduces random variables in Sarsa from $\{s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}\}$ to $\{s_t, a_t, r_{t+1}, s_{t+1}\}$.

TD learning of action values - n-step Sarsa

The definition of action value is

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a].$$

The discounted return G_t can be written in different forms as

$$\begin{aligned}
\text{Sarsa} \quad \longleftarrow \quad & G_t^{(1)} = R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}), \\
& G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2}, A_{t+2}), \\
& \quad \vdots \\
n\text{-step Sarsa} \quad \longleftarrow \quad & G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}), \\
& \quad \vdots \\
\text{MC} \quad \longleftarrow \quad & G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots
\end{aligned}$$

It should be noted that $G_t = G_t^{(1)} = G_t^{(2)} = G_t^{(n)} = G_t^{(\infty)}$, where the superscripts merely indicate the different decomposition structures of G_t.

- **Sarsa aims to solve**

$$q_\pi(s, a) = \mathbb{E}[G_t^{(1)} | s, a] = \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | s, a].$$

- **MC learning aims to solve**

$$q_\pi(s, a) = \mathbb{E}[G_t^{(\infty)} | s, a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots | s, a].$$

- **An intermediate algorithm called n-step Sarsa aims to solve**

$$q_\pi(s, a) = \mathbb{E}[G_t^{(n)} | s, a] = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n}, A_{t+n}) | s, a].$$

- **The algorithm of n-step Sarsa is**

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t) \left[q_t(s_t, a_t) - [r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})] \right].$$

n-step Sarsa is more general because it becomes the (one-step) Sarsa algorithm when $n = 1$ and the MC learning algorithm when $n = \infty$.

TD learning of optimal action values: Q-learning

Q-learning algorithm

$$
\begin{aligned}
q_{t+1}(s_t, a_t) &= q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)]\right], \\
q_{t+1}(s, a) &= q_t(s, a), \quad \forall (s, a) \neq (s_t, a_t),
\end{aligned}
$$

The difference between Sarsa and Q_learning in TD Target:

- **The TD target in Q-learning is** $r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)$.
- **The TD target in Sarsa is** $r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$.

即，Sarsa 仅仅是选择最优的那个 a，而 Q-learning 是同时会优化这个 a！

Mathematically, it solves a Bellman optimality equation expressed in action values:

$$
q(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1}, a) \middle| S_t = s, A_t = a\right], \quad \forall s, a.
$$

Off-policy & On-policy

Concepts:

- The **behavior policy** is used to generate experience samples.
- The **target policy** is constantly updated toward an optimal policy.

On-policy vs off-policy:

- **When the behavior policy is the same as the target policy**, such kind of learning is called **on-policy**.
- **When they are different**, the learning is called **off-policy**.

Advantages of off-policy learning:

- It can search for optimal policies based on the experience samples generated by any other polices!
  - 最直接的好处就是，比如说 behavior policy 探索性很强，那么我们就可以从其获得的 trajectory 中很好的估计出每一个 state-action-pair 对应的 action value；
  - 相反，如果比如用 target policy 生成 trajectory，然后再用其估计，因为 target policy 可能是 greedy 的，或者 ε-greedy 的，探索性很弱，所以就很难探索到所有的 s 和 a，进而估计出所有 (s, a) 的最优值

How to judge

1. Check what the algorithm does mathematically
2. Check what things are required to implement the algorithm. It deserves special attention because it is one of the most confusing problems.

Sarsa is on-policy.

- First, Sarsa aims to solve the Bellman equation of a given policy π:

$$q_\pi(s,a) = \mathbb{E}\left[R + \gamma q_\pi(S', A')|s,a\right], \quad \forall s, a.$$

  where $R \sim p(R|s,a)$, $S' \sim p(S'|s,a)$, $A' \sim \pi(A'|S')$.

- Second, the algorithm is

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\Big[q_t(s_t,a_t) - [r_{t+1} + \gamma q_t(s_{t+1},a_{t+1})]\Big],$$

  which requires $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$:

  - If (s_t, a_t) is given, then r_{t+1} and s_{t+1} do not depend on any policy!
  - a_{t+1} is generated following $\pi_t(s_{t+1})$!

- π_t is both the target and behavior policy.

Monte Carlo learning is on-policy.

- First, the MC method aims to solve

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma R_{t+2} + \ldots | S_t = s, A_t = a\right], \quad \forall s, a.$$

  where the sample is generated following a given policy π.

- Second, the implementation of the MC method is

$$q(s,a) \approx r_{t+1} + \gamma r_{t+2} + \ldots$$

- A policy is used to generate samples, which is further used to estimate the action values of the policy. Based on the action values, we can improve the policy.

Q-learning is off-policy.

- First, Q-learning aims to solve the Bellman optimality equation

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_a q(S_{t+1},a) \middle| S_t = s, A_t = a\right], \quad \forall s, a.$$

- Second, the algorithm is

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - [r_{t+1} + \gamma \max_{a\in\mathcal{A}} q_t(s_{t+1},a)]\right]$$

  which requires $(s_t, a_t, r_{t+1}, s_{t+1})$.
  - If (s_t, a_t) is given, then r_{t+1} and s_{t+1} do not depend on any policy!
- The behavior policy to generate a_t from s_t can be anything. The target policy will converge to the optimal policy.

Pseudocode

Off - policy

Pseudocode: Optimal policy search by Q-learning (off-policy version)

For each episode $\{s_0, a_0, r_1, s_1, a_1, r_2, \dots\}$ generated by π_b, do

For each step $t = 0, 1, 2, \dots$ of the episode, do

Update q-value:

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\Big[q(s_t,a_t) - [r_{t+1} + \gamma \max_a q_t(s_{t+1},a)]\Big]$$

Update target policy:

$\pi_{T,t+1}(a|s_t) = 1$ if $a = \arg\max_a q_{t+1}(s_t,a)$

$\pi_{T,t+1}(a|s_t) = 0$ otherwise

- There are two policies here: one is π_b, serving as the **behavior policy** to generate data; the other is π_T, serving as the **target policy** that ultimately converges to the optimal policy.
- The reason we do not use an ϵ-greedy policy here is that π_T is no longer required to generate data. The original purpose of adopting ϵ-greedy was to utilize its inherent randomness for data generation!

On - Policy

Pseudocode: Policy searching by Q-learning (on-policy version)

For each episode, do

If the current s_t is not the target state, do

Collect the experience $(s_t, a_t, r_{t+1}, s_{t+1})$: In particular, take action a_t following $\pi_t(s_t)$, generate r_{t+1}, s_{t+1}.

Update q-value:

$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\Big[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)]\Big]$

Update policy:

$\pi_{t+1}(a|s_t) = 1 - \frac{\epsilon}{|\mathcal{A}|}(|\mathcal{A}| - 1)$ if $a = \arg\max_a q_{t+1}(s_t, a)$

$\pi_{t+1}(a|s_t) = \frac{\epsilon}{|\mathcal{A}|}$ otherwise

A unified point of view

All the algorithms we introduced in this lecture can be expressed in a unified expression:

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)[q_t(s_t, a_t) - \bar{q}_t],$$

where $\bar{q}_t$ is the *TD target*.
Different TD algorithms have different $\bar{q}_t$.

Algorithm	Expression of $\bar{q}_t$
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1}, a_{t+1})$
n-step Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n}, a_{t+n})$
Expected Sarsa	$\bar{q}_t = r_{t+1} + \gamma \sum_a \pi_t(a\|s_{t+1}) q_t(s_{t+1}, a)$
Q-learning	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1}, a)$
Monte Carlo	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \ldots$

All the algorithms can be viewed as stochastic approximation algorithms solving the **Bellman equation** or **Bellman optimality equation**.

Algorithm	Equation aimed to solve
Sarsa	BE: $q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \middle| S_t = s, A_t = a\right]$
n-step Sarsa	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(s_{t+n}, a_{t+n}) \middle| S_t = s, A_t = a]$
Expected Sarsa	BE: $q_\pi(s,a) = \mathbb{E}\big[R_{t+1} + \gamma \mathbb{E}_{A_{t+1}}[q_\pi(S_{t+1}, A_{t+1})] \big| S_t = s, A_t = a\big]$
Q-learning	BOE: $q(s,a) = \mathbb{E}\left[R_{t+1} + \max_a q(S_{t+1}, a) \middle| S_t = s, A_t = a\right]$
Monte Carlo	BE: $q_\pi(s,a) = \mathbb{E}[R_{t+1} + \gamma R_{t+2} + \ldots \mid S_t = s, A_t = a]$